Project Report
On
MedScribe AI

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Vinay Kuthey	EN22CS3011084
2	Viraj Agrawal	EN22CS3011089
3	Lakshita Singh	EN22CS3011135
4	Vishakha Shadija	EN22ME304114
5	Vishakha Hilsayan	EN22EL301065

Group Name: 09D10

Project Number: GAI-22

Industry Mentor Name:

University Mentor Name: Prof. Divya Kumawat

Academic Year: 2026

Table of Contents

1. Problem Statement & Objectives

1.1 Introduction
1.2 Problem Statement
1.3 Project Objectives
1.4 Scope of the Project

2. Proposed Solution

2.1 Key features
2.2 Overall Architecture / Workflow
2.3 Tools & Technologies Used

3. Results & Output

3.1 Screenshots / outputs
3.2 Key outcomes

4. Conclusion

5. Future Scope & Enhancements

1. Problem Statement & Objectives

1.1 Introduction

The MedScribe AI is a GenAI-based application designed to reduce the repetitive, time-consuming tasks i.e. generate structured medical documents such as patient summaries and medical reports. The system uses Retrieval-Augmented Generation (RAG) by combining a vector database (ChromaDB in-memory) with a Large Language Model (Gemini 2.5 Flash API). It automates this using three technologies working together: Prompt Engineering to instruct the LLM on clinical structure and tone, ChromaDB as a Vector Database implementing RAG so that past cases enrich future generations, and Google Gemini 2.5 Flash as the free-tier LLM that produces the actual document. It ensures consistent formatting, professional tone, and domain-specific terminology by implementing instruction-based prompt engineering to generate precise response.

1.2 Problem Statement

Build a specialized GenAI content generation tool tailored for Healthcare professionals. By implementing sophisticated Prompt Engineering techniques, the application enables users to transform basic topics into high-quality professional outputs, such as 'Patient Summaries'. The system ensures consistency in tone, formatting, and industry-standard terminology, significantly reducing the manual effort required for drafting complex documentation.

1.3 Project Objectives

- Automate the generation of structured clinical documents from minimal input.
- Apply Prompt Engineering to enforce medical terminology, tone, and formatting.
- Implement Retrieval-Augmented Generation (RAG) using ChromaDB as the Vector DB.
- Provide a clean, form-based web interface for healthcare professionals.
- Using Google Gemini 2.5 Flash (free tier) as the underlying LLM.
- Make the RAG process transparent to the user through a status indicator showing when past cases were retrieved and used.

1.4 Scope of the Project

MedScribe AI is demonstrating the practical integration of three Al engineering components - Prompt Engineering, Vector DB with RAG, and LLM API, in a healthcare documentation context.

- Minimal Interface: A simple and minimal interface to enter input data and generate desired output.
- Backend Logic: Gemini 2.5 Flash with ChromaDB im-memory providing top 2 similar past cases and injected into prompt.
- Document types Supported:

  - Patient Summary — SOAP-format structured note (Subjective, Objective, Assessment, Plan)
  - Medical Report — Formal detailed diagnostic report with clinical findings and prognosis.

2. Proposed Solution

2.1 Key Features

- Structured patient input form with 7 dedicated fields ensuring complete and well-organised data collection.
- Two document type options — Patient Summary (SOAP format) and Medical Report.
- Role-based Prompt Engineering templates that enforce clinical terminology, formatting, and professional tone.
- RAG architecture using ChromaDB — every generated document is embedded and stored for future context retrieval.
- Dynamic few-shot prompting — past similar cases are automatically retrieved and injected into the prompt.
- Real-time RAG status indicator showing whether Vector DB context was used, with expandable context preview.
- One-click copy of generated documents to clipboard.
- Zero-cost deployment using free-tier Gemini API, open-source ChromaDB, and open-source Flask.

2.2 Overall Architecture/ Workflow

The system follows a layered architecture consisting of three main layers: Frontend Layer, Backend Layer, and AI & Data Layer. It implements a Retrieval-Augmented Generation (RAG) pipeline to generate accurate and structured healthcare content.

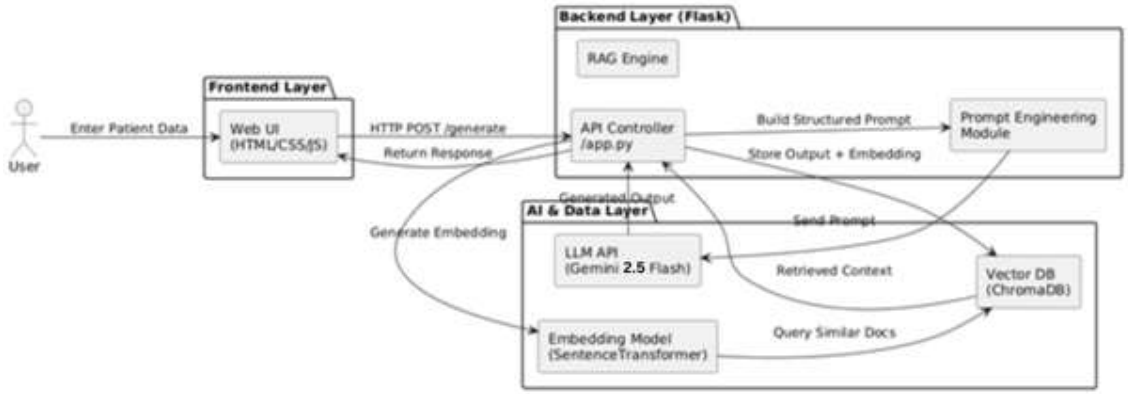


Fig. Architecture Diagram

Frontend Layer (User Interface)

- Built using HTML, CSS, and JavaScript
- Allows users (healthcare professionals) to input patient details
- Sends data to backend via HTTP POST (/generate)
- Displays the generated patient summary

Backend Layer (Flask)

- Acts as the core processing unit
- Handles API requests through the API Controller (app.py)
- Manages the RAG pipeline workflow
- Coordinates between embedding model, vector database, and LLM
- Uses a Prompt Engineering Module to:
    - Structure output in SOAP format
    - Maintain medical terminology and consistency

AI & Data Layer

This layer provides intelligence and context-aware generation:

- **Embedding Model (SentenceTransformer):**
  - Converts input text into vector embeddings
- **Vector Database (ChromaDB):**
  - Stores embeddings and generated documents
  - Retrieves similar past records for context (RAG)
- **LLM API (Gemini 2.5 Flash):**
  - Generates structured patient summaries
  - Uses both user input and retrieved context

Information Flow

- Input Data: Patient case description
- Processed Data: Embeddings (vector format)
- Context Data: Retrieved similar documents
- Output Data: Structured patient summary (SOAP format)

Flow:

User Input → Embedding → Vector DB → Context → Prompt → LLM → Output → Storage

Activity Workflow

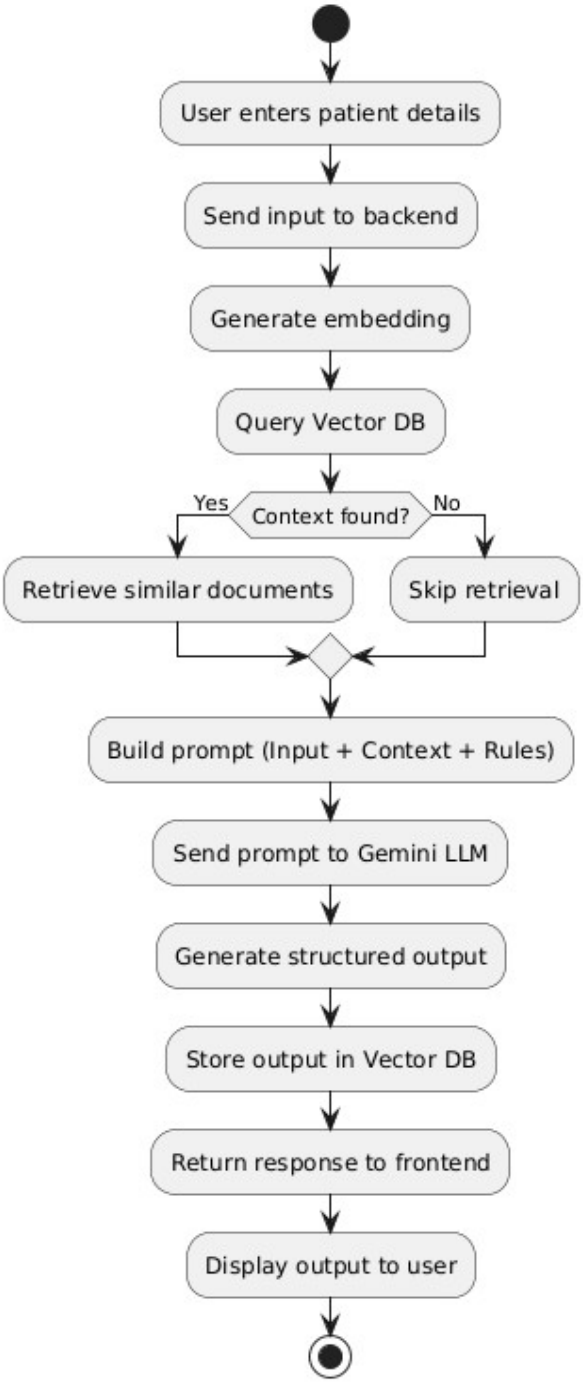


Fig. Activity Diagram

Workflow Description:

User Input:
The process starts when the user enters patient details in the frontend.

Send to Backend:
The input is sent to the Flask backend via an API request.

Embedding Generation:
The backend converts the input text into vector embeddings using the embedding model.

Vector DB Query:
The system queries the vector database to find similar past documents.

Decision – Context Found?:

- **Yes:** Relevant documents are retrieved as context
- **No:** Retrieval step is skipped

Prompt Construction:
A structured prompt is created using:

- User input
- Retrieved context (if available)
- Predefined rules (SOAP format, medical terminology)

LLM Processing:
The prompt is sent to the Gemini LLM for content generation.

Output Generation:
The LLM generates a structured patient summary.

Store Output:
The generated output is stored in the vector database for future retrieval.

Return Response:
The backend sends the response back to the frontend and is displayed to the user.

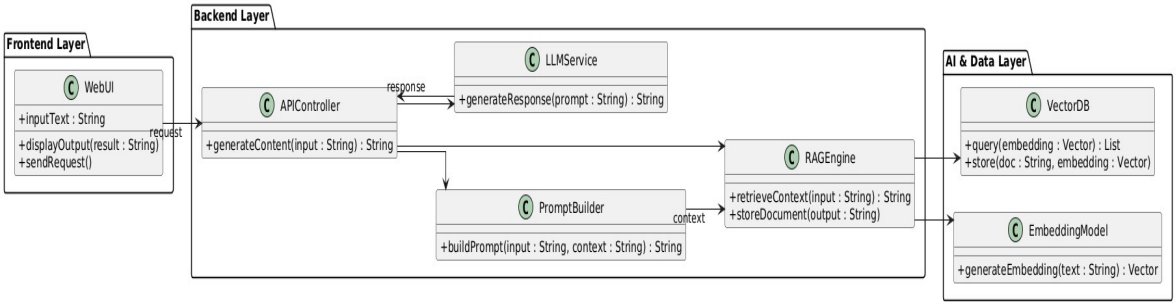


Fig. Class Diagram

Relationships Between Classes

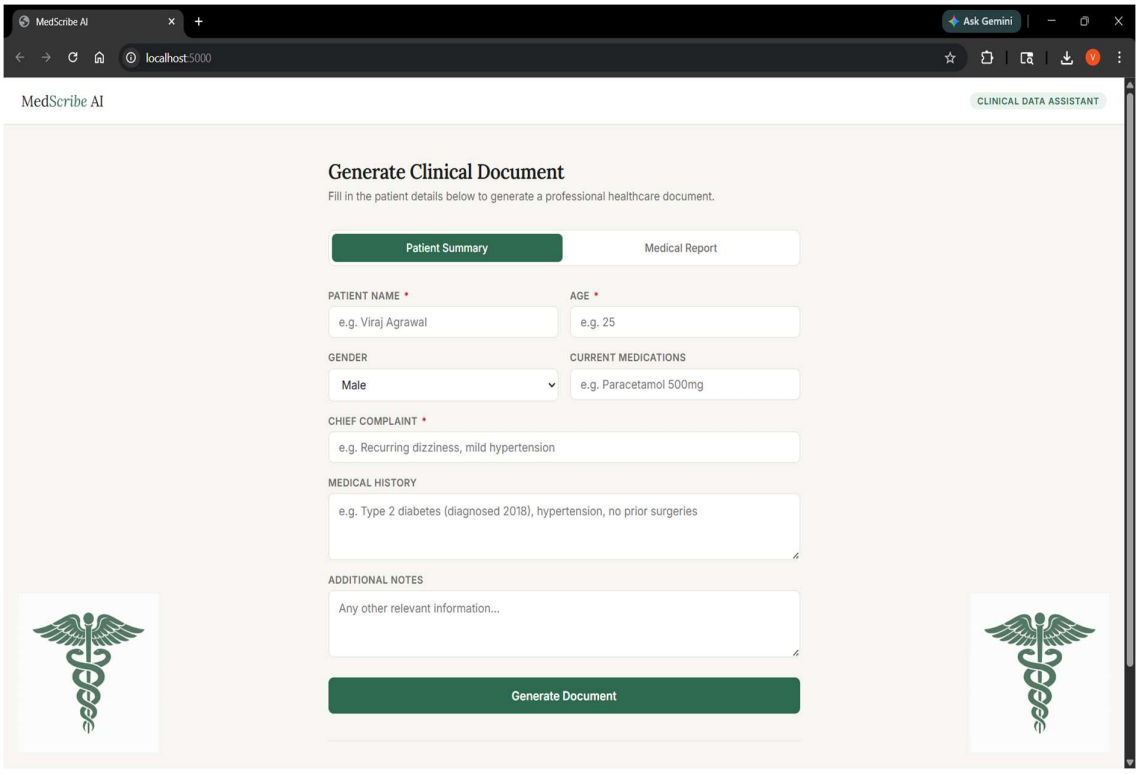
- **WebUI → APIController:** sends user request
- **APIController → RAGEngine:** handles retrieval & storage
- **APIController → PromptBuilder:** constructs prompt
- **APIController → LLMService:** generates output
- **RAGEngine → VectorDB & EmbeddingModel:** for retrieval and storage

2.3 Tools & Technologies

Tool/ Technology	Role in Project
Python	Core backend Programming language
Flask	Web server framework- serves HTML, routing and API endpoints
GenAI SDK + Gemini 2.5 Flash	SDK for calling GenAI LLM from python for generating structured clinical documents.
ChromaDB	VectorDB- stores and retrieves document embedding for RAG
HTML+CSS+JS	Frontend Interface- Form, output rendering & RAG status

3. Result & Output

3.1 Screenshots/Output



Main interface

3.2 Key Outcomes

The key outcomes obtained after implementing the project are Structured document generation, RAG implementation, Input accordance, Prompt Engineering., Zero cost deployment and dynamic few shot transition and even the accessibility to one click copy the output.

4. Conclusion

MedScribe Al successfully demonstrates the practical application of three core Al engineering concepts — Prompt Engineering, Retrieval-Augmented Generation with a Vector Database, and LLM API integration — in a real-world healthcare documentation context.

The system reduces manual effort for healthcare professionals by automatically generating structured, terminology-consistent clinical documents from minimal input. The RAG layer ensures that the tool's output quality improves with each use, as past cases enrich the context provided to the LLM.

The clean, form-based interface keeps complexity hidden from the user while exposing just enough transparency (the RAG status badge and expandable context) to build trust in the Al-generated output.

5. Future Scope & Enhancements

- Switch to chromadb.PersistentClient for data persistence across restarts.
- Integrate a real embedding model (e.g., sentence-transformers) for better RAG.
- Add user authentication and HIPAA-compliant data handling.
- Support additional document types: Discharge Notes, Clinical Referrals, Prescription Summaries.
- Export generated documents as PDF or Word files directly from the Ul.
- Add a history panel to review and search all past generated documents.